```
      .oauth2Login(Customizer.withDefaults())
      .logout(logout -> logout.logoutSuccessHandler(
        oidcLogoutSuccessHandler(clientRegistrationRepository)))
      .build();
  }
}
```

❶ Allows unauthenticated access to the SPA static resources

❷ Allows unauthenticated read access to the books in the catalog

❸ Any other request requires user authentication.

Let’s test how Edge Service works now. Ensure that the Polar UI, Redis, and Keycloak containers are still running. Next, build and run the Edge Service application (`./gradlew bootRun`), and then go to http://localhost:9000 from an incognito browser window. The first thing to notice is that you don’t get redirected to the login page but are presented with the Angular frontend application immediately. You can start the authentication flow by clicking the Login button in the upper-right menu.

After logging in, the upper-right menu will contain a Logout button, which is only shown if the current user is successfully authenticated. Click the button to log out. It should trigger the logout flow, but it won’t work due to CSRF issues. You’ll learn how to fix that in the next section. In the meantime, stop the application with Ctrl-C.

11.5.3 Protecting against Cross-Site Request Forgery

The interaction between the frontend and backend is based on a session cookie. After the user is successfully authenticated with the OIDC/OAuth2 strategy, Spring will generate a session identifier to match the authenticated context and send it to the browser as a cookie. Any subsequent request to the backend will have to contain the session cookie, from which Spring Security